

go-sqlite3 / _example / trace / main.go

...



mattn fix type of event code

324c3f7 · 8 years ago



264 lines (220 loc) · 6.24 KB

Code

Blame

Raw



```
1 package main
2
3 import (
4     "database/sql"
5     "fmt"
6     "log"
7     "os"
8
9     sqlite3 "github.com/mattn/go-sqlite3"
10 )
11
12 func traceCallback(info sqlite3.TraceInfo) int {
13     // Not very readable but may be useful; uncomment next line in case of doubt:
14     //fmt.Printf("Trace: %#v\n", info)
15
16     var dbErrText string
17     if info.DBError.Code != 0 || info.DBError.ExtendedCode != 0 {
18         dbErrText = fmt.Sprintf("; DB error: %#v", info.DBError)
19     } else {
20         dbErrText = "."
21     }
22
23     // Show the Statement-or-Trigger text in curly braces ('{', '}')
24     // since from the *paired* ASCII characters they are
25     // the least used in SQL syntax, therefore better visual delimiters.
26     // Maybe show 'ExpandedSQL' the same way as 'StmtOrTrigger'.
27     //
28     // A known use of curly braces (outside strings) is
29     // for ODBC escape sequences. Not likely to appear here.
30     //
31     // Template languages, etc. don't matter, we should see their *result*
32     // at *this* level.
33     // Strange curly braces in SQL code that reached the database driver
34     // suggest that there is a bug in the application.
35     // The braces are likely to be either template syntax or
36     // a programming language's string interpolation syntax.
```

```

37
38     var expandedText string
39     if info.ExpandedSQL != "" {
40         if info.ExpandedSQL == info.StmtOrTrigger {
41             expandedText = " = exp"
42         } else {
43             expandedText = fmt.Sprintf(" expanded {%q}", info.ExpandedSQL)
44         }
45     } else {
46         expandedText = ""
47     }
48
49     // SQLite docs as of September 6, 2016: Tracing and Profiling Functions
50     // https://www.sqlite.org/c3ref/profile.html
51     //
52     // The profile callback time is in units of nanoseconds, however
53     // the current implementation is only capable of millisecond resolution
54     // so the six least significant digits in the time are meaningless.
55     // Future versions of SQLite might provide greater resolution on the profiler callback.
56
57     var runTimeText string
58     if info.RunTimeNanosec == 0 {
59         if info.EventCode == sqlite3.TraceProfile {
60             //runTimeText = "; no time" // seems confusing
61             runTimeText = "; time 0" // no measurement unit
62         } else {
63             //runTimeText = "; no time" // seems useless and confusing
64         }
65     } else {
66         const nanosPerMillisec = 1000000
67         if info.RunTimeNanosec%nanosPerMillisec == 0 {
68             runTimeText = fmt.Sprintf("; time %d ms", info.RunTimeNanosec/nanosPerMi
69         } else {
70             // unexpected: better than millisecond resolution
71             runTimeText = fmt.Sprintf("; time %d ns!!!", info.RunTimeNanosec)
72         }
73     }
74
75     var modeText string
76     if info.AutoCommit {
77         modeText = "-AC-"
78     } else {
79         modeText = "+Tx+"
80     }
81
82     fmt.Printf("Trace: ev %d %s conn 0x%x, stmt 0x%x {%q}%s%s%s\n",
83         info.EventCode, modeText, info.ConnHandle, info.StmtHandle,
84         info.StmtOrTrigger, expandedText,
85         runTimeText,
86         dbErrText)
87     return 0
88 }
89

```

```

90  ✓ func main() {
91      eventMask := sqlite3.TraceStmt | sqlite3.TraceProfile | sqlite3.TraceRow | sqlite3.Trace
92
93      sql.Register("sqlite3_tracing",
94          &sqlite3.SQLiteDriver{
95              ConnectHook: func(conn *sqlite3.SQLiteConn) error {
96                  err := conn.SetTrace(&sqlite3.TraceConfig{
97                      Callback:      traceCallback,
98                      EventMask:      eventMask,
99                      WantExpandedSQL: true,
100                  })
101                  return err
102              },
103          })
104
105      os.Exit(dbMain())
106  }
107
108  // Harder to do DB work in main().
109  // It's better with a separate function because
110  // 'defer' and 'os.Exit' don't go well together.
111  //
112  // DO NOT use 'log.Fatal...' below: remember that it's equivalent to
113  // Print() followed by a call to os.Exit(1) --- and
114  // we want to avoid Exit() so 'defer' can do cleanup.
115  // Use 'log.Panic...' instead.
116
117  ✓ func dbMain() int {
118      db, err := sql.Open("sqlite3_tracing", ":memory:")

```

```
191         defer stmt.Close()
192
193         for i := 0; i < nGenRows; i++ {
194             result, err := stmt.Exec(textPrefix + Descr)
195             if err != nil {
```

```

196             log.Panic(err)
197         }
198
199         resultDoCheck(result, Descr, i)
200     }
201 }
202
203 ✓ func resultDoCheck(result sql.Result, callerDescr string, callIndex int) {
204     lastID, err := result.LastInsertId()
205     if err != nil {
206         log.Panic(err)
207     }
208     nAffected, err := result.RowsAffected()
209     if err != nil {
210         log.Panic(err)
211     }
212
213     log.Printf("Exec result for %s (%d): ID = %d, affected = %d\n", callerDescr, callIndex,
214 }
215
216 ✓ func dbDoSelect(db *sql.DB) {
217     const Descr = "DB-Query"
218
219     rows, err := db.Query(selectDML, noteTextPattern)
220     if err != nil {
221         log.Panic(err)
222     }
223     defer rows.Close()
224
225     rowsDoFetch(rows, Descr)
226 }
227
228 ✓ func dbDoSelectPrepared(db *sql.DB) {
229     const Descr = "DB-Prepare"
230
231     stmt, err := db.Prepare(selectDML)
232     if err != nil {
233         log.Panic(err)
234     }
235     defer stmt.Close()
236
237     rows, err := stmt.Query(noteTextPattern)
238     if err != nil {
239         log.Panic(err)
240     }
241     defer rows.Close()
242
243     rowsDoFetch(rows, Descr)
244 }
245
246 ✓ func rowsDoFetch(rows *sql.Rows, callerDescr string) {
247     var nRows int
248     var id int64

```

```
248     var id int64
249     var note string
250
251     for rows.Next() {
252         err := rows.Scan(&id, &note)
253         if err != nil {
254             log.Panic(err)
255         }
256         log.Printf("Row for %s (%d): id=%d, note=%q\n",
257             callerDescr, nRows, id, note)
258         nRows++
259     }
260     if err := rows.Err(); err != nil {
261         log.Panic(err)
262     }
263     log.Printf("Total %d rows for %s.\n", nRows, callerDescr)
264 }
```